Mochi — A Baseline-Aware PC Health Buddy

Operating Systems Mini Project · Concept document · 16 October 2026

The idea in a nutshell

Mochi is a desktop companion that learns what "normal" looks like on *your* PC, then tells you in plain words when something is off, why it's happening, and what to do about it.

It lives in the system tray as a small, squishy rice-cake mascot whose mood reflects the health of the machine. Mochi puffs up when RAM is full, sweats when the CPU runs hot, gets dizzy when the system starts thrashing, and naps when everything is idle.

Under the cute surface, Mochi does three serious jobs:

1. **Learn a personal baseline.** It builds a model of normal CPU, memory, disk and temperature behaviour for each time of day and each kind of activity.
2. **Detect and explain anomalies.** It flags usage that is well above your normal, identifies the process responsible, and diagnoses the underlying OS-level cause (swapping, I/O wait, throttling, a leak).
3. **Notice when the PC is underperforming.** It tracks app startup times and runs small benchmarks against the PC's own history, and suggests a guided PC check when performance slips.

Problem statement

Existing tools like Task Manager and htop show raw numbers, but they don't tell the user whether those numbers are normal or what to do about them.

- **Fixed thresholds don't fit real people.** A rule like "warn at 80% CPU" spams a gamer or a student compiling code, yet stays silent when a light user's idle laptop sits at 40% because of a runaway background process.
- **The real cause is often invisible.** A PC can feel sluggish with CPU at 30%, because the actual problem is memory pressure (swapping), disk I/O wait or thermal throttling. Ordinary monitors don't surface these.
- **Slow decline goes unnoticed.** Startup bloat, a filling disk, a slow memory leak or a degrading drive make the PC a little slower each week. Nobody notices until it's painful.
- **Warnings without advice are useless.** "High memory usage" tells the user nothing about which app is responsible or what action would help.

Users need a monitor that compares against *their own* normal behaviour, explains the OS-level cause of a problem in plain language, and gives one clear, actionable tip.

Objectives

1. Collect system-wide and per-process resource metrics (CPU, memory, swap, disk I/O, temperature, CPU frequency) every few seconds with low overhead (target: under 2% CPU for Mochi itself).
2. Learn contextual baselines for each user and detect sustained abnormal usage with statistical methods (EWMA and z-scores) instead of fixed thresholds.
3. Diagnose the likely OS-level cause of each anomaly (memory pressure, I/O bottleneck, thermal throttling, runaway process, memory leak) and attribute it to the responsible process.
4. Detect task-level underperformance through app startup timing and periodic micro-benchmarks, and trigger a guided PC check when performance degrades.
5. Present all of this through a friendly, non-alarming interface with clear notifications and one actionable tip per alert.

System architecture

Mochi is a pipeline of small components connected by a thread-safe queue, so collecting data never waits on analysis or the UI.

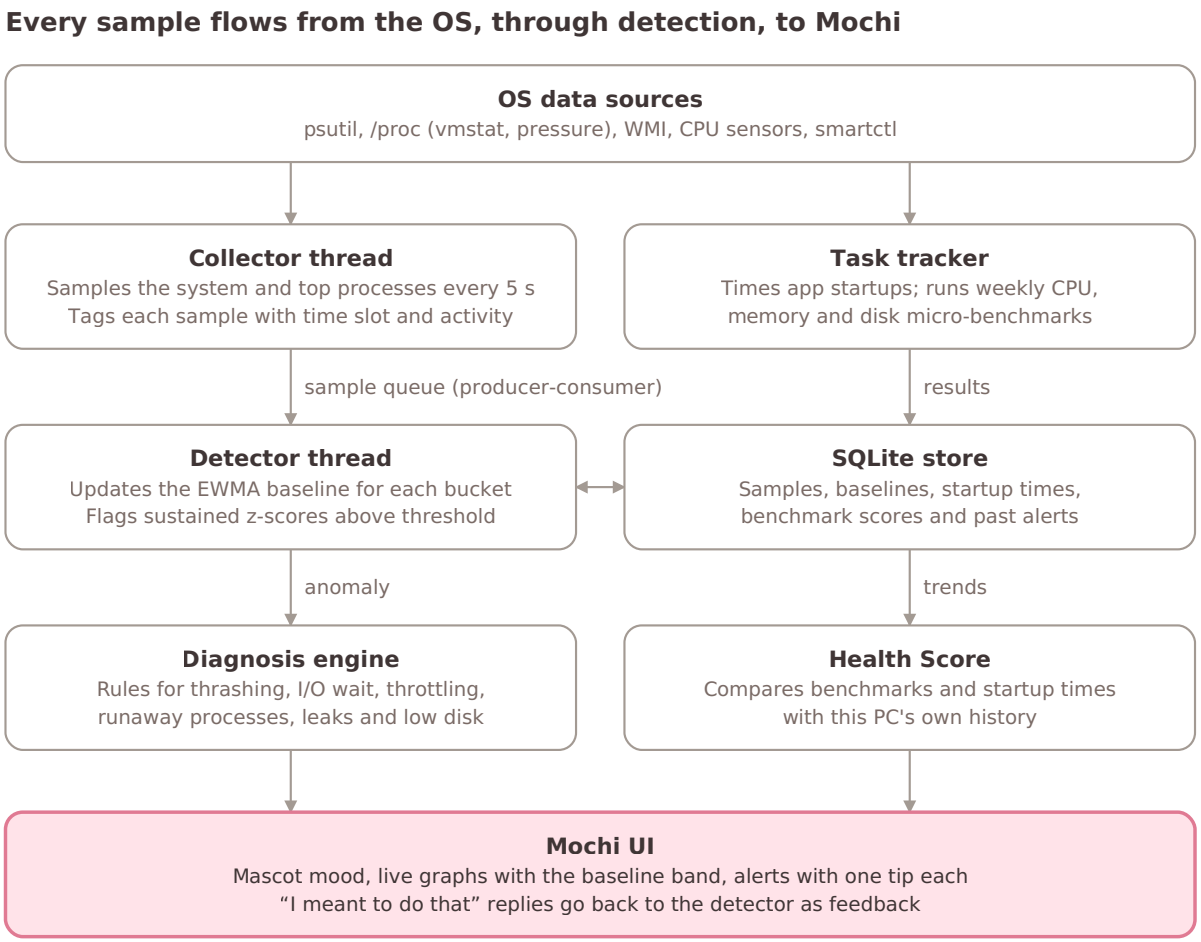


Figure 1. Mochi architecture: seven components, producer-consumer between collector and detector.

The collector produces samples and the detector consumes them (a classic producer-consumer pair), while the task tracker works on its own schedule. Everything is persisted in SQLite, so baselines survive restarts and the dashboard can show history.

Baseline learning and anomaly detection

Mochi judges every reading against what is normal *in that context*, so 70% CPU while gaming at 9 pm is fine but 70% at 3 am on an idle desktop is flagged.

Contextual buckets. Each metric gets a separate baseline per bucket, where a bucket is a pair of:

- **Time slot:** hour of day, split into weekday vs weekend (48 slots).
- **Activity class:** what the user is doing, inferred from the foreground app. Classes: idle, browsing, coding, gaming, media, office, other. A small editable mapping file assigns apps to classes.

Adaptive mean and variance (EWMA). For each bucket, Mochi keeps an exponentially weighted moving average and variance, so the baseline slowly follows changing habits without storing all history:

$$\mu_t = \alpha \cdot x_t + (1 - \alpha) \cdot \mu_{t-1} \qquad \sigma_t^2 = (1 - \alpha) \cdot (\sigma_{t-1}^2 + \alpha \cdot (x_t - \mu_{t-1})^2)$$

A small alpha (around 0.01–0.05) makes the baseline stable; a larger one makes it adapt faster.

Z-score. How unusual a reading is, measured in standard deviations from the bucket's normal:

$$z_t = (x_t - \mu_t) / \sigma_t$$

Sustained-duration rule. A single spike is never an anomaly. Mochi raises an alert only when z stays above a threshold for a minimum time:

Level	Condition	Mochi says
Psst	z > 2 for 2 minutes	A quiet badge on the tray icon
Heads up	z > 3 for 5 minutes	A notification with cause and tip
Uh-oh	z > 4 for 5 minutes, or an absolute limit hit (disk > 95% full, temp > 95 °C)	A notification that stays until dismissed

Cold start. For the first 3 days Mochi is in "learning mode": it uses only conservative absolute limits and shows a progress bar ("Getting to know your PC: 62%"). A bucket needs at least 30 samples before z-scores are trusted.

User feedback loop. Every alert has an "I meant to do that" button. Choosing it tags that period as intentional (for example a video render), so it is excluded from the baseline and similar episodes are not flagged again.

Pseudocode (main detection loop):

```
every 5 seconds:
    sample = collect_metrics()                  # system-wide + top processes
    bucket = (time_slot(now), activity_class(foreground_app()))
    store(sample, bucket)                       # SQLite

    for metric in [cpu, ram, swap, disk_io, temp]:
        b = baseline[bucket][metric]
        if b.samples < 30:
            check_absolute_limits(metric, sample); continue
        z = (sample[metric] - b.mean) / max(b.std, MIN_STD)
        streak[metric] = streak[metric] + 5 if z > Z_ALERT else 0
        if streak[metric] >= SUSTAIN_SECONDS:
            cause = diagnose(metric, sample)    # see Diagnosis engine
            notify(level(z), cause, tip_for(cause))
            streak[metric] = 0
        if not is_marked_intentional(now):
            b.update_ewma(sample[metric])
```

Diagnosis engine

When an anomaly fires, Mochi looks past the symptom to the OS-level cause, using a set of rules checked in order. The first rule that matches decides the explanation and the tip.

Symptom	Evidence Mochi checks	Likely cause	Mochi's tip
Sluggish, but CPU is not maxed	Swap in use rising, major page faults far above baseline, RAM > 90%	Memory pressure / thrashing	"Your PC is shuffling memory to disk. Closing Chrome (5.8 GB) would help most."
High CPU	One process using > 50% of total CPU for 5+ min; run-queue length above core count	Runaway process	"python.exe has been at 98% for 12 minutes. Is it stuck?"
Everything lags, CPU looks low	High I/O wait %, disk busy time near 100%, long disk queue	Disk I/O bottleneck	"Your disk is the bottleneck. OneDrive is syncing 3 GB; pause it for now."
Slows down after minutes of load	CPU frequency well below max while temperature is high	Thermal throttling	"I'm overheating, so the CPU is slowing itself down. Check the vents and use a flat surface."
RAM slowly climbing for hours	One process's memory grows steadily with no drops (positive linear trend)	Memory leak	"Discord has grown by 900 MB in 3 hours and never let go. A restart will fix it."
Disk almost full	Free space < 10% or shrinking fast	Storage exhaustion	"Only 8 GB left. Want me to show the biggest folders?"
High CPU at idle	High usage while no user input for 10+ min	Background task or unwanted software	"Something is busy while you're away: updater.exe. Here's what it is."
Battery draining fast	Discharge rate above baseline for this activity	Power-hungry app	"Battery is draining 2x faster than usual. Spotify is the top power user."

Each rule outputs three things: the cause in plain words, the culprit process (if any), and a tip with a one-click action where safe (open the app's folder, open startup settings, end process after confirmation).

The process names and figures in the tips are examples of the format; real alerts fill them in from live data.

Task underperformance and the PC check

Mochi decides a PC is underperforming by comparing it to its own past performance, using two signals.

1. App startup timing. When a tracked app launches, Mochi notes its process creation time and measures how long until its CPU usage settles (a practical "ready" signal). Each app keeps a rolling median of its startup time.

- Example: "VS Code usually opens in 2.1 s. Today it took 7.8 s."
- A single slow launch is ignored; a trend is not (median of the last 5 launches more than 50% above the 30-day median).

2. Micro-benchmarks. Three tiny, repeatable tests that take about 10 seconds in total. They run weekly while the PC is idle and plugged in, or on demand from the Check-up screen.

Test	What it does	What it reveals
CPU	A fixed hashing loop, timed	Throttling, background load
Memory	Copies a large buffer repeatedly, timed	Memory pressure, bandwidth
Disk	Writes and reads a 256 MB temp file with fsync, timed	Drive health, I/O contention

Scores are combined into a **Health Score out of 100**, relative to the PC's first two weeks.

When the PC check is suggested. If the Health Score drops below 80, or startup times trend up across several apps, Mochi suggests a check-up: "I've been feeling a bit slow lately. Want to do a check-up together?"

The guided PC check walks through a checklist and marks each item as fine, worth a look, or needs action:

- ☐ Free disk space and the largest folders
- ☐ Disk health (SMART status where available)
- ☐ Programs that launch at startup, and how much each delays boot
- ☐ CPU temperature under a short load, and whether throttling occurs
- ☐ Background processes using resources while idle
- ☐ Long-running processes with growing memory (possible leaks)
- ☐ Battery health and charge cycles (laptops)

The check ends with a short report card and Mochi's top 3 suggestions.

Fun quirks

The quirks make Mochi delightful to use, but each one is backed by a real metric, so they double as features in the report.

Mochi's moods. The mascot's face and shape are a live summary of system health:

Mood	Triggered by	What Mochi looks like
Happy	Everything within baseline	Round, gentle smile, slow breathing animation
Sleepy	No user input for 10+ min, low usage	Eyes closed, floating "z" letters
Puffy	RAM well above baseline	Inflated and round, cheeks puffed
Sweaty	High temperature or throttling	Little sweat drops, fanning itself
Dizzy	Thrashing (heavy swapping)	Spiral eyes, wobbling
Grumpy	Disk nearly full	Frowning, squished flat
Zoomies	A benchmark is running	Bouncing quickly
Proud	Health Score improved or a hog was closed	Sparkles, tiny crown

Quirks list:

- **"Who ate my RAM?" detective mode.** One click shows a suspect line-up: the top memory users as cards with mugshot-style app icons, memory used, and how long each has been growing. The guilty one gets a "prime suspect" stamp.
- **Daily PC weather report.** A one-line forecast each morning from yesterday's data: "Mostly sunny, with a 70% chance of Chrome by evening."
- **OS fact of the day.** Real counters from the OS, made fun: "Today your CPU switched between tasks 4.2 million times and handled 1.1 million interrupts." (Context switches and interrupts are real values from the kernel.)
- **Feed Mochi.** When the user closes a resource hog after a tip, the freed memory becomes snacks. Freeing 2 GB earns a whole dango; Mochi visibly enjoys it.
- **"I meant to do that" button.** Teaches Mochi that a heavy period was intentional. Mochi replies, "Got it, you're rendering. I'll stay quiet." This is also the baseline feedback loop.
- **Focus / Shh mode.** Mochi automatically holds non-urgent alerts while a full-screen app runs (games, presentations) and delivers a summary afterwards.
- **Optional gentle roasts.** Off by default. "47 tabs open. Are you collecting them?"
- **Achievements.** Badges such as *Tab Tamer* (stayed under your RAM baseline all week), *Cool Cucumber* (no throttling for 7 days), *Clean Sweep* (freed 10 GB of disk), *Speed Demon* (new fastest boot).
- **Calm streaks.** A counter of days without an Uh-oh alert, shown as a small row of rice cakes.
- **Mochi's diary.** The weekly report, written as a diary entry in Mochi's voice: "Monday was rough. Someone opened Premiere and Chrome together and I got very puffy."
- **Poke Mochi.** Clicking the mascot makes it squish and shows a 3-line status bubble.
- **Personal records.** Fastest boot, longest uptime, lowest idle RAM, shown like a high-score board.

OS concepts covered

Mochi covers six of the seven syllabus modules directly, which makes the report's "OS concepts" section easy to justify.

Module	Concept	Where it appears in Mochi
1. Introduction to OS	OS services, resource management	Reading system metrics through OS interfaces (/proc, WMI, psutil)
2. OS Principles	Processes, PCB, threads, system calls	Per-process attribution; collector, detector and UI run as separate threads
3. Scheduling	Run queue, load average, context switches	Diagnosing CPU contention; OS fact of the day
4. Concurrency	Synchronization, producer-consumer	Collector thread produces samples into a queue; detector thread consumes them; locks around the shared baseline
5. Memory Management	Virtual memory, paging, swapping, thrashing, working set	Memory-pressure diagnosis via page faults and swap; leak detection
7. Storage, Protection & Security	Disk I/O, disk management, protection	I/O-wait diagnosis, disk benchmark, SMART checks; flagging unknown background processes

Module 6 (virtualization and file systems) is touched lightly through the disk-space analysis in the PC check.

Tech stack

Python keeps the project achievable in a few weeks, with optional C for the benchmark to show low-level work.

Layer	Choice	Why
Language	Python 3.11+	Fast to build; psutil covers both Windows and Linux
Metrics	psutil; on Linux also /proc/vmstat (page faults), /proc/pressure (PSI stall %), lm-sensors; on Windows WMI	Deep metrics for the diagnosis engine
Storage	SQLite	Sample history and baselines; already in the recommended tools
GUI	PySide6 (Qt) with QSS stylesheets, or pywebview with HTML/CSS	Both allow rounded, custom, animated UI; Tkinter makes the cute look hard
Charts	pyqtgraph (with PySide6) or Chart.js (with pywebview)	Smooth live graphs with a shaded baseline band
Notifications	plyer, or Qt's system-tray messages	Cross-platform desktop alerts
Concurrency	threading + queue.Queue	Producer-consumer between collector and detector
Benchmarks	Python, optionally a small C module	Repeatable timing tests
Disk health	smartctl (smartmontools)	SMART status in the PC check
Testing	unittest, plus stress scripts (stress-ng on Linux)	Unit tests and fault injection

Layer	Choice	Why
Version control	Git and GitHub	Required good practice

Platform note: Linux gives far richer metrics (pressure stall information, page faults, I/O wait) and is the better choice for the diagnosis engine. On Windows, temperatures may need LibreHardwareMonitor.

Scope and timeline

Build the core first so there is always a working demo, then add stretch features and quirks as time allows.

Core (must have for the demo):

- Collector, SQLite storage and contextual baselines
- Sustained z-score anomaly detection with the three alert levels
- Diagnosis engine with at least the memory-pressure, runaway-process, disk-I/O and leak rules
- Notifications with cause and tip, plus the "I meant to do that" feedback
- Home and Timeline screens with Mochi's moods and the baseline band

Stretch (if time allows):

- Startup-time tracker, micro-benchmarks and the Health Score
- Guided PC check with report card
- Detective mode, diary, achievements, weather report and other quirks
- Thermal-throttling and battery rules (need laptop sensors)

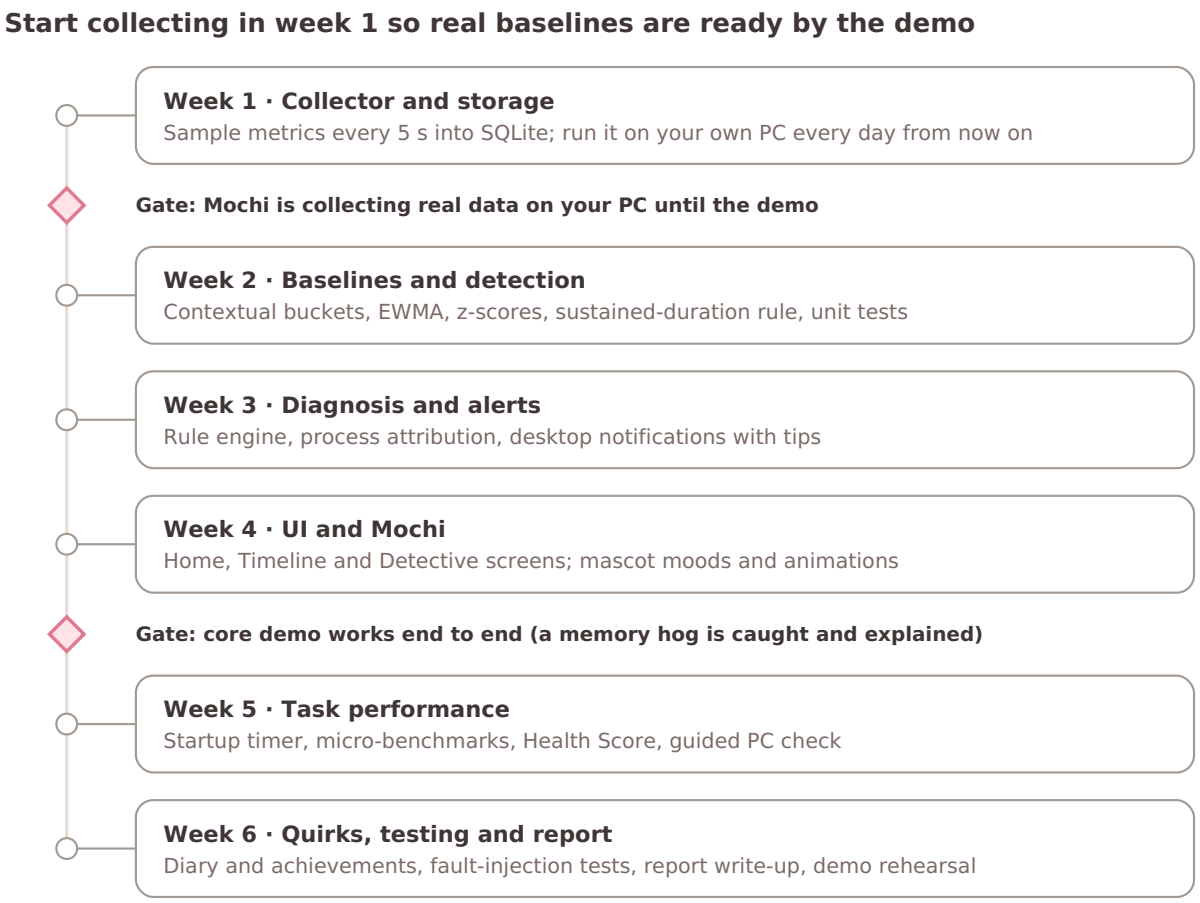


Figure 2. Six-week build plan with two gates.

The plan assumes about six weeks; compress weeks 5 and 6 if your deadline is closer, since the core demo is already complete at the second gate.

Testing and demo plan

Mochi is validated by deliberately causing each problem and checking that it is caught, correctly diagnosed, and attributed to the right process.

Unit tests cover the EWMA and z-score maths on synthetic data, the sustained-duration rule (a 10-second spike must not alert; a 6-minute plateau must), and each diagnosis rule with hand-made metric snapshots.

Fault-injection tests:

Test	How it is induced	Expected Mochi reaction
Memory pressure	Script allocates RAM until the system swaps	Puffy, then Dizzy; "memory pressure" diagnosis naming the script
Runaway process	Infinite CPU loop on all cores	"Runaway process" alert within ~5 minutes
Memory leak	Script grows by 50 MB per minute, never frees	Leak alert naming the script, with growth rate
Disk bottleneck	Large continuous file copy	"Disk I/O bottleneck" diagnosis

Test	How it is induced	Expected Mochi reaction
Disk filling	Create large dummy files until free space < 10%	Grumpy; disk-space tip
Thermal throttling	Sustained full load (stress-ng) on a laptop	Sweaty; throttling diagnosis if frequency drops
Normal heavy use	Gaming or compiling during a known busy slot	No alert (baseline already covers it)

False-alarm rate. Run Mochi on your own PC for one normal week and count alerts. Report how many were real and how many were false; the target is fewer than one false alert per day.

Overhead. Measure Mochi's own CPU and RAM use; target under 2% CPU and under 100 MB RAM.

Live demo script (about 5 minutes):

1. Start collecting data on your own PC 1–2 weeks before the demo so real baselines exist.
2. Open the dashboard: show the shaded "normal band" on the graphs and a happy Mochi.
3. Run the memory hog script. Mochi puffs up, gets dizzy, and the alert names the script with the right cause.
4. Open detective mode and show the suspect line-up.
5. Close the hog: Mochi gets fed and turns proud.
6. Run the PC check-up and show the Health Score and report card.
7. Finish with Mochi's diary from the past week.

UI design language: "Soft Serve"

Mochi's interface should feel like a pastel Japanese snack shop: soft, round, warm and calm. Even a warning should feel like a friend tapping your shoulder, not an alarm going off.

Principles

1. **Calm by default.** Most of the time the screen is quiet and happy. Strong colour is saved for things that need attention.
2. **Explain, then suggest.** Every alert follows the same pattern: what's happening, why, and one thing to do.
3. **Soft everything.** No sharp corners, harsh lines or pure black.
4. **Mochi is the status light.** The mascot's mood is the fastest way to read system health; numbers are one click away.
5. **Cute, never confusing.** Every playful element has a plain-language label for clarity and accessibility.

Colour palette

Token	Name	Light mode	Dark mode (“Midnight Mochi”)	Used for
`background`	Rice Cream	#FFF8F0	#1E1B2E	App background
`surface`	Mochi White	#FFFFFF	#2A2640	Cards and panels

Token	Name	Light mode	Dark mode (“Midnight Mochi”)	Used for
`text`	Cocoa	#4A3F3F	#F3EDE7	Main text (never pure black)
`text-muted`	Milk Tea	#8C7B75	#A79FB8	Secondary text, labels
`accent`	Strawberry Milk	#FFB5C2	#FF9FB2	Buttons, highlights, Mochi's blush
`info`	Lavender	#C9B8F0	#A996E6	Info and Psst level
`good`	Matcha	#A8D5BA	#7FC39A	Normal, healthy, the baseline band
`warning`	Mango	#FFD38A	#F5BE5C	Heads-up level
`critical`	Cherry	#FF8A8A	#FF7676	Uh-oh level (soft red, not alarm red)

All text and background combinations should meet WCAG AA contrast (4.5:1); pastel colours are used for fills and backgrounds, never for small text.

Typography

- **Headings:** Nunito (rounded sans), weight 700–800.
- **Body:** Nunito, weight 400–600, 14–15 px.
- **Numbers and stats:** a rounded font with tabular figures (for example Nunito's tabular numerals) so live values don't jiggle as they update.
- Sentence case everywhere; no all-caps shouting.

Shapes and layout

- Corner radius: 20 px on cards, 12 px on inner elements, fully round (pill) buttons and chips.
- Cards float on the background with a very soft, warm shadow (cocoa at 6–8% opacity, 16 px blur) instead of borders.
- Generous spacing on an 8 px grid; one idea per card.
- Gauges are soft rings or "jelly" bars with rounded ends, not sharp speedometers.

The mascot

- Mochi is built from simple shapes: a squashed rounded blob, two dot eyes, a small curved mouth and pink blush ovals. This keeps it easy to draw in SVG or Qt and to animate.
- Each mood (see Fun quirks) changes only a few features: eye shape, mouth, body squash, and one small prop (sweat drop, "z" letters, spiral eyes, crown).
- Mochi sits on the Home screen, in every notification, and as the tray icon (simplified to just the face at small sizes).
- Mochi is drawn originally for this project; no existing characters are used.

Graphs

- The baseline appears as a soft Matcha-tinted band ("your normal") behind the live line.
- The live line is smooth and rounded, in Cocoa or Lavender.
- Anomalies are marked with a small star or Mochi-face marker; hovering explains them.

- Area fills use low opacity (15–20%) to keep graphs gentle.

Motion

- **Breathing:** Mochi gently scales 100% to 103% on a 3-second loop when idle.
- **Squash and stretch:** on clicks and mood changes, a quick squish and bounce (about 250 ms, ease-out-back).
- **Transitions:** cards fade and rise 8 px when they appear; nothing flashes or shakes.
- Respect the OS "reduce motion" setting by turning off loops and keeping only fades.

Notifications

Every notification has the same parts:

1. Mochi's face showing the matching mood.
2. A short title by level: "Psst...", "Heads up!" or "Uh-oh."
3. One line on the cause: "Chrome is using 5.8 GB, about twice your usual."
4. One tip: "Closing a few tabs would help."
5. Two pill buttons: a primary action ("Show me") and "I meant to do that".

Voice and copy

- Mochi speaks in the first person, warmly and briefly: "I'm feeling a bit puffy" rather than "Memory threshold exceeded."
- Never blames the user, never uses scary words like "critical failure".
- Always follows a playful line with a precise one: the cute part sets the tone, the number gives the facts.
- Technical detail (z-score, page faults/sec) lives behind a "Nerd stats" toggle for those who want it, which is also handy during the demo.

Screens

Screen	What it shows
Home	Big Mochi with today's mood, the weather report line, three soft gauges (CPU, RAM, disk)
Timeline	Live and historical graphs with the baseline band and anomaly markers
Detective	The "Who ate my RAM?" suspect line-up of top processes
Check-up	Health Score, benchmark results, PC check checklist and report card
Diary	Weekly diary entries, achievements, streaks and personal records
Settings	Learning progress, sensitivity slider ("chill" to "watchful"), roasts toggle, app-to-activity mapping